CHMLTECH ONLINE JOURNAL · VOLUME 104, ISSUE 2 · RESEARCH ARTICLE

Multi-Agent AI Systems: Architectures, Coordination Patterns and Failure Modes

Quak Foo Lee
Chief Technology Officer, Chmltech Ltd.
Division of Artificial Intelligence and Machine Learning · Vancouver, BC, Canada

Published November 9, 2026 18 min read

ABSTRACT

Systems in which several large language model (LLM) agents cooperate have become easy to build with frameworks such as AutoGen, CAMEL, MetaGPT and ChatDev, but understanding of when they outperform a single agent has lagged behind. This paper reviews LLM-based multi-agent systems from an engineering perspective. It relates them to classical multi-agent research, including the agent properties of Wooldridge and Jennings, the contract net protocol and blackboard architectures, and describes the planning, memory and tool capabilities of individual agents. Five coordination topologies are compared: orchestrator-worker, hierarchical, peer and debate, blackboard, and sequential pipeline. The evidence for benefits, including multi-agent debate and parallel research agents, is weighed against findings that strong single agents and sampling-and-voting ensembles often perform comparably and that intrinsic self-correction is weak. Failure modes are examined through the Multi-Agent System Failure Taxonomy, which identifies fourteen modes in three categories. Simple quantitative models are derived for token multiplication, which grows quadratically with the number of agents in broadcast debate, for parallel latency, and for compounding error in sequential pipelines with and without

verification. Security risks from agent-to-agent prompt injection are discussed. The paper concludes with criteria for when not to use multiple agents and with design guidelines emphasizing explicit roles, structured hand-offs, externally grounded verification, bounded loops and least privilege.

Keywords: multi-agent systems; LLM agents; agent coordination; multi-agent debate; failure taxonomy; prompt injection; agent orchestration

1. Introduction

Multi-agent systems have a long history in artificial intelligence, but the arrival of capable large language models (LLMs) has changed both what an agent is and how agents are assembled into systems. Instead of hand-coded reasoning components exchanging messages in a formal agent communication language, contemporary systems connect several instances of an LLM, each given a role, instructions, tools and memory, and let them coordinate in natural language. Frameworks such as AutoGen make it straightforward to define conversable agents and the patterns by which they interact [1], and role-based systems such as MetaGPT encode human organizational procedures into agent workflows [2].

The ease of construction has outpaced understanding of when such systems help. Cemri et al. observed that, despite enthusiasm for multi-agent LLM systems, their performance gains on popular benchmarks are often minimal, and they built the first empirically grounded taxonomy of why such systems fail [3]. At the same time, multi-agent designs multiply token consumption and latency and introduce new security exposures, since every agent that reads another agent's output is a potential recipient of injected instructions. Practitioners therefore need a principled basis for choosing a topology, estimating its cost and reliability, and deciding when a single agent is the better design.

This paper provides such a basis. Section 2 defines LLM-based multi-agent systems in relation to the classical literature and describes the anatomy of an individual agent. Section 3 classifies coordination topologies. Section 4 reviews representative frameworks. Section 5 assesses the evidence for benefits, including evidence against. Section 6 examines failure modes and evaluation. Section 7 derives simple models of cost, latency and reliability. Section 8 treats security. Section 9 states when multiple agents should not be used and offers design guidelines, and Section 10 concludes.

2. Definitions and Agent Anatomy

2.1 Classical multi-agent systems

Wooldridge and Jennings characterized an intelligent agent by four properties under a weak notion of agency: autonomy, operating without direct intervention and with control over its own actions and state; social ability, interacting with other agents through some communication language; reactivity, perceiving and responding to its environment in a timely way; and pro-activeness, exhibiting goal-directed behavior by taking the initiative [4]. A multi-agent system is then a collection of such agents whose interactions determine the behavior of the whole. Classical research produced coordination mechanisms that remain relevant. The contract net protocol of Smith casts task allocation as negotiation, in which a manager announces a task, contractors bid and the manager awards the contract [5]. The blackboard architecture, exemplified by the Hearsay-II speech-understanding system, coordinates independent knowledge sources through a shared data structure on which partial hypotheses are posted and refined [6].

2.2 LLM-based agents and multi-agent systems

Surveys of LLM-based agents describe an architecture in which an LLM serves as the core controller and is augmented with modules for profile or role, memory, planning and action [7, 8]. A survey of LLM-based multi-agent systems organizes the field by the environment in which agents operate, how agents are profiled, how they communicate and how their capabilities are acquired or improved [9]. For the purposes of this paper, an LLM-based multi-agent system is defined as a system in which two or more LLM-driven components, each with a distinct role, context or tool set, exchange messages or shared state in order to accomplish a task that the system receives as a whole. The definition excludes a single agent that calls tools, and it also excludes simple repeated sampling from one prompt, although Section 5 shows that the latter is an important baseline.

The classical properties map imperfectly onto LLM agents. Social ability is strong, since natural language is a flexible communication medium. Autonomy is real but bounded by the context window and by the instructions that the system designer provides. Pro-activeness depends heavily on prompting and on the orchestration code that decides when an agent runs. Most importantly, LLM agents built on the same base model are far less independent

than distinct human experts or hand-engineered components; they share training data, biases and blind spots, which limits the diversity that multi-agent designs are intended to exploit.

2.3 Anatomy of an individual agent

Three capabilities recur. **Planning** decomposes a goal into steps and revises the plan in light of observations. ReAct interleaves reasoning traces with actions so that each observation conditions the next thought [10], and Reflexion has an agent write verbal reflections on feedback from failed attempts and store them in an episodic memory that conditions subsequent trials [11]. **Memory** extends the agent beyond its context window. Generative Agents maintain a memory stream of natural-language records of experience, retrieve records by recency, importance and relevance, and synthesize them into higher-level reflections that inform planning; ablations showed that observation, planning and reflection each contributed to the believability of agent behavior [12]. **Tools** connect the agent to external systems through function calls, search, code execution or databases [7]. In a multi-agent system each of these capabilities may be assigned to different agents, which is precisely what creates coordination problems.

3. Coordination Topologies

The topology of a multi-agent system determines who may communicate with whom, who holds control and where state resides. Five patterns account for most published and deployed systems. They are summarized in Table 1 and described below. Figure 1 draws the five patterns side by side; the main differences to notice are where control sits and whether agents read each other's output directly, through a shared workspace, or only through a lead agent.

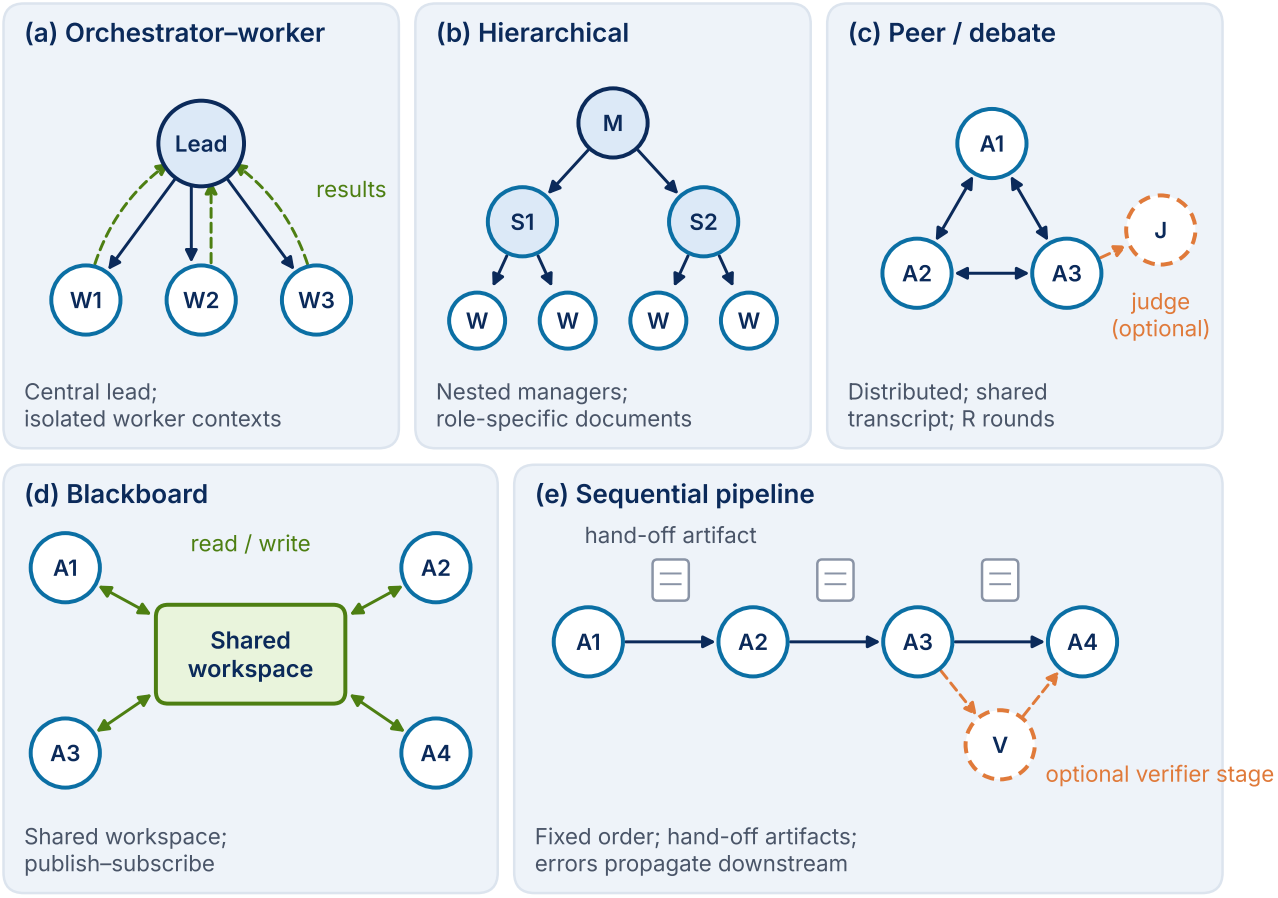


Figure 1. The five coordination topologies of Table 1: (a) orchestrator–worker, (b) hierarchical, (c) peer and debate with an optional judge, (d) blackboard and (e) sequential pipeline with an optional verifier stage. Arrows show the direction of messages or hand-offs; dashed elements are optional.

Drawn by the author from Table 1 and the works cited there.

3.1 Orchestrator-worker

A lead agent decomposes the task, dispatches subtasks to worker agents, often in parallel, and synthesizes their results. Workers typically have independent, clean contexts and do not communicate with one another. Anthropic described a research system of this type in which a lead agent plans the research and spawns subagents that search in parallel, each with its own context window [13]. The pattern resembles the classical contract net without bidding [5]. Its strengths are parallelism and context isolation; its weakness is that the orchestrator is a single point of failure whose decomposition quality bounds the result.

3.2 Hierarchical

Hierarchical systems extend orchestration recursively, with managers supervising sub-managers. MetaGPT assigns roles modeled on a software company, such as product manager, architect, engineer and quality-assurance engineer, and passes structured documents between them according to standardized operating procedures [2]. Hierarchy mirrors human organizations and localizes responsibility but adds layers through which information is summarized and potentially lost.

3.3 Peer and debate

In peer topologies agents communicate as equals. In multi-agent debate, several instances propose answers, read one another's responses and revise over multiple rounds until they converge [14]. Liang et al. added a judge that manages the debate and assigned agents opposing stances to counter what they called the degeneration-of-thought problem, in which a single model that has become confident in an answer fails to generate new ideas through self-reflection [15]. CAMEL uses a two-agent role-playing scheme with inception prompting, in which an AI user and an AI assistant cooperate on a task with limited human input [16].

3.4 Blackboard

Agents read from and write to a shared workspace rather than addressing messages to each other, following the Hearsay-II tradition [6]. MetaGPT's shared message pool, to which agents publish structured outputs and subscribe according to their roles, is a modern instance of the idea [2]. Blackboards decouple agents and make state inspectable, but they require discipline about schemas and write permissions, since every agent can be influenced by everything posted.

3.5 Sequential pipeline

A fixed sequence of agents each transforms the output of its predecessor. ChatDev organizes software development as a chat chain through design, coding and testing phases, in which pairs of agents converse within each phase [17]. Pipelines are simple to reason about and to test stage by stage, but they are vulnerable to compounding error, quantified in Section 7, because a defect introduced early propagates downstream.

Table 1. Comparison of coordination topologies for LLM-based multi-agent systems.

Topology	Control and state	Strengths	Weaknesses	Representative work
Orchestrator-worker	Central lead; isolated worker contexts	Parallelism; context isolation; clear synthesis point	Lead is a bottleneck and single point of failure; high token use	[13]
Hierarchical	Nested managers; role-specific documents	Mirrors organizations; localized responsibility	Information loss across layers; added latency	[2]
Peer / debate	Distributed; shared transcript	Error correction through critique; diverse candidate answers	Token cost grows quickly with agents and rounds; convergence on shared errors	[14, 15, 16]
Blackboard	Shared workspace; publish–subscribe	Loose coupling; inspectable state	Contamination of shared state; schema discipline required	[6, 2]
Sequential pipeline	Fixed order; hand-off artifacts	Simple to test and audit stage by stage	Compounding error; no recovery without verifier stages	[17]

Characterization by the author from the cited works.

4. Representative Frameworks

Several research frameworks established the vocabulary of the field. AutoGen provides customizable, conversable agents that can combine LLMs, human input and tools, with interaction behaviors programmed in natural language or code; its authors demonstrated applications in mathematics, coding, question answering, operations research, online

decision-making and entertainment [1]. CAMEL introduced role-playing with inception prompting and used it to generate conversational data for studying cooperative agent behavior [16]. MetaGPT encodes standardized operating procedures into prompt sequences and uses an assembly-line division of roles; its authors argued that structured intermediate outputs reduce the cascading hallucinations that arise from naively chaining LLMs [2]. ChatDev guides what agents communicate through its chat chain and how they communicate through a mechanism it calls communicative dehallucination [17].

Other frameworks address composition and optimization. AgentVerse dynamically assembles groups of expert agents and studied emergent behaviors in collaboration [18]. GPTSwarm represents agents as computational graphs whose nodes query LLMs or process data and whose edges carry information, and it optimizes both node prompts and graph connectivity automatically [19]. Generative Agents, although designed for social simulation rather than task solving, demonstrated how memory, reflection and planning allow 25 agents in a sandbox to produce coordinated emergent behavior, such as spreading invitations to a party from a single seeded intention [12]. Table 2 summarizes the frameworks.

Table 2. Representative LLM multi-agent frameworks and their coordination mechanisms.

Framework	Primary topology	Key mechanism	Demonstrated domain
AutoGen	Configurable conversation patterns	Conversable agents mixing LLMs, humans and tools	Mathematics, coding, question answering, decision-making
CAMEL	Two-agent peer role-play	Inception prompting	Cooperative task solving; data generation
MetaGPT	Hierarchical with shared message pool	Standardized operating procedures; structured outputs	Software engineering
ChatDev	Sequential chat chain	Phase-wise dual-agent dialogue; communicative dehallucination	Software engineering
AgentVerse	Dynamic expert groups	Recruitment and adjustment of group composition	Multiple task types
GPTSwarm	Optimizable graph	Automatic node and edge optimization	Multiple benchmarks
Generative Agents	Peer society in a sandbox	Memory stream, reflection and planning	Social behavior simulation

Source: [1, 16, 2, 17, 18, 19, 12].

5. Evidence for and against Multi-Agent Benefits

5.1 Evidence for benefits

Du et al. reported that having multiple LLM instances propose, debate and revise answers over several rounds significantly improved mathematical and strategic reasoning and the factual validity of generated content across their tasks, using identical procedures and prompts for all tasks [14]. Liang et al. reported that debate with a judge was effective on commonsense machine translation and counter-intuitive arithmetic, while also finding that an adaptive stopping rule and a modest degree of disagreement were required, and that an LLM

may not be a fair judge when different LLMs serve as debaters [15]. In an industrial setting, Anthropic reported that its orchestrator-worker research system outperformed a single-agent configuration by 90.2% on an internal research evaluation, a result specific to that system, task class and metric [13].

5.2 The ensembling baseline

Much of the measured benefit may derive from ensembling rather than from interaction. Self-consistency samples multiple reasoning paths from one model and selects the most frequent final answer, improving chain-of-thought reasoning without any agent communication [20]. Li et al. found that performance scales with the number of agents under a simple sampling-and-voting method, that this gain is orthogonal to more elaborate methods, and that the degree of improvement correlates with task difficulty [21]. Under the idealized assumption that each of N agents (N odd) answers a binary question correctly and independently with probability *p*, the majority is correct with probability

$$P_{\text{maj}} = \Sigma_{k=(N+1)/2}^{N} C(N, k)\, p^{k}\, (1 - p)^{N-k} \tag{1}$$

where C(N, k) is the binomial coefficient. For p = 0.7 and N = 5, Eq. (1) gives $P_{\text{maj}} \approx 0.837$. The formula is an upper bound on what voting can achieve in practice, because instances of the same model make correlated errors; when errors are perfectly correlated, voting yields no gain at all. Eq. (1) also shows that voting amplifies errors when p < 0.5, so ensembling is no remedy for tasks the base model usually gets wrong.

5.3 Evidence against

Wang et al. re-evaluated multi-agent discussion systematically and found that a single LLM agent with strong prompts achieved almost the same performance as the best existing discussion approach across a wide range of reasoning tasks and backbone models; discussion outperformed a single agent only when the prompt contained no demonstrations [22]. Huang et al. showed that LLMs struggle to correct their own reasoning without external feedback and that performance sometimes degrades after intrinsic self-correction [23]. Because critique by another instance of the same model is a form of intrinsic feedback, this finding cautions against assuming that adding reviewer agents improves correctness unless the reviewers have access to external signals such as test results, retrieved evidence or tool

outputs. Cemri et al. likewise note that gains of multi-agent systems on popular benchmarks are often minimal [3].

> A fair comparison for any multi-agent design is a single agent given the same total token budget, strong prompts and the same tools, together with a simple sampling-and-voting ensemble. Published gains that do not control for these baselines should be interpreted cautiously.

6. Failure Modes and Evaluation

6.1 A taxonomy of failures

Cemri et al. constructed the Multi-Agent System Failure Taxonomy (MAST) from expert analysis of 150 execution traces, validated it with high inter-annotator agreement ($\kappa = 0.88$), and applied it to MAST-Data, a dataset of 1,642 annotated traces collected from seven multi-agent frameworks across coding, mathematics and general agent tasks [3]. The taxonomy identifies 14 failure modes in three categories: system design issues, inter-agent misalignment and task verification, as listed in Table 3. The authors also developed an LLM-as-a-judge annotation pipeline with high agreement with human annotators to scale the analysis [3].

Table 3. Failure modes in the Multi-Agent System Failure Taxonomy (MAST) and corresponding design responses.

Category	Failure modes	Design responses
FC1 System design issues	1.1 Disobey task specification; 1.2 Disobey role specification; 1.3 Step repetition; 1.4 Loss of conversation history; 1.5 Unaware of termination conditions	Precise role and task specifications; explicit stopping criteria; step limits; durable state outside the context window
FC2 Inter-agent misalignment	2.1 Conversation reset; 2.2 Fail to ask for clarification; 2.3 Task derailment; 2.4 Information withholding; 2.5 Ignored other agent's input; 2.6 Reasoning–action mismatch	Structured hand-off schemas; required clarification steps; shared task objective restated at each hand-off; action validation against stated plan
FC3 Task verification	3.1 Premature termination; 3.2 No or incomplete verification; 3.3 Incorrect verification	Verification against external signals (tests, retrieval, rules); independent verifier with defined acceptance criteria

Failure modes from [3]; design responses synthesized by the author.

Two findings are especially relevant to design. First, verification failures appear frequently even in runs that succeed, indicating that apparent success does not imply that the system checked its work [3]. Second, targeted interventions helped but did not solve the problem: in the authors' ChatDev case study, improving role specifications yielded a 9.4% increase in task success, and adding a high-level objective verification step yielded a 15.6% improvement on their ProgramDev task set, yet task completion rates remained low and the effectiveness of tactical prompt and topology changes was not consistent across case studies [3]. The implication is that many failures are failures of system design, analogous to poor organizational design in human teams, rather than deficiencies of the underlying model alone.

6.2 Evaluating multi-agent systems

Evaluation should measure the outcome of the whole system under realistic conditions together with process-level diagnostics. Outcome metrics include task success against a ground truth or rubric, total token consumption, wall-clock latency and the frequency of

human intervention. Process diagnostics classify failures in each trace using a taxonomy such as MAST [3]. Automated judges can scale both, but Zheng et al. documented position, verbosity and self-enhancement biases in LLM judges even where agreement with humans was high [24]; judges should therefore be calibrated against human labels. Because multi-agent behavior is stochastic, results should be reported over repeated runs with variance, and comparisons should include the single-agent and ensembling baselines of Section 5.

7. Cost, Latency and Reliability

7.1 Token multiplication

Multi-agent systems consume substantially more tokens than single interactions. Anthropic reported that, in its data, agents typically used about four times more tokens than chat interactions and multi-agent systems about fifteen times more, and that token usage alone explained 80% of performance variance on the BrowseComp evaluation [13]. The mechanism can be made explicit for a broadcast debate. Let N agents participate in R rounds; in each round every agent reads the task prompt of T_0 tokens plus all responses from previous rounds, each of length L tokens, and produces a response of length L. The total token count is

$$T_{\text{total}} = N\,R\,T_0 + N^2\,L\,R(R-1)/2 + N\,R\,L \tag{2}$$

where the first two terms are input tokens and the last is output tokens. The quadratic dependence on N arises because each agent reads every other agent's history. For N = 3, R = 3, T_0 = 1,000 and L = 500, Eq. (2) gives 9,000 + 13,500 + 4,500 = 27,000 tokens, compared with 1,500 tokens for a single agent answering once, an eighteen-fold increase. The figure is illustrative and not directly comparable to the empirical ratios above, which concern different system designs. Orchestrator-worker designs avoid the quadratic term by giving workers isolated contexts and returning only condensed results to the lead, at the risk of losing information that the synthesis step needs. Figure 2 plots Eq. (2) for these parameters: the cross-reading term already supplies half of the tokens at N = 3 and dominates as agents are added, which is the term that isolated worker contexts remove.

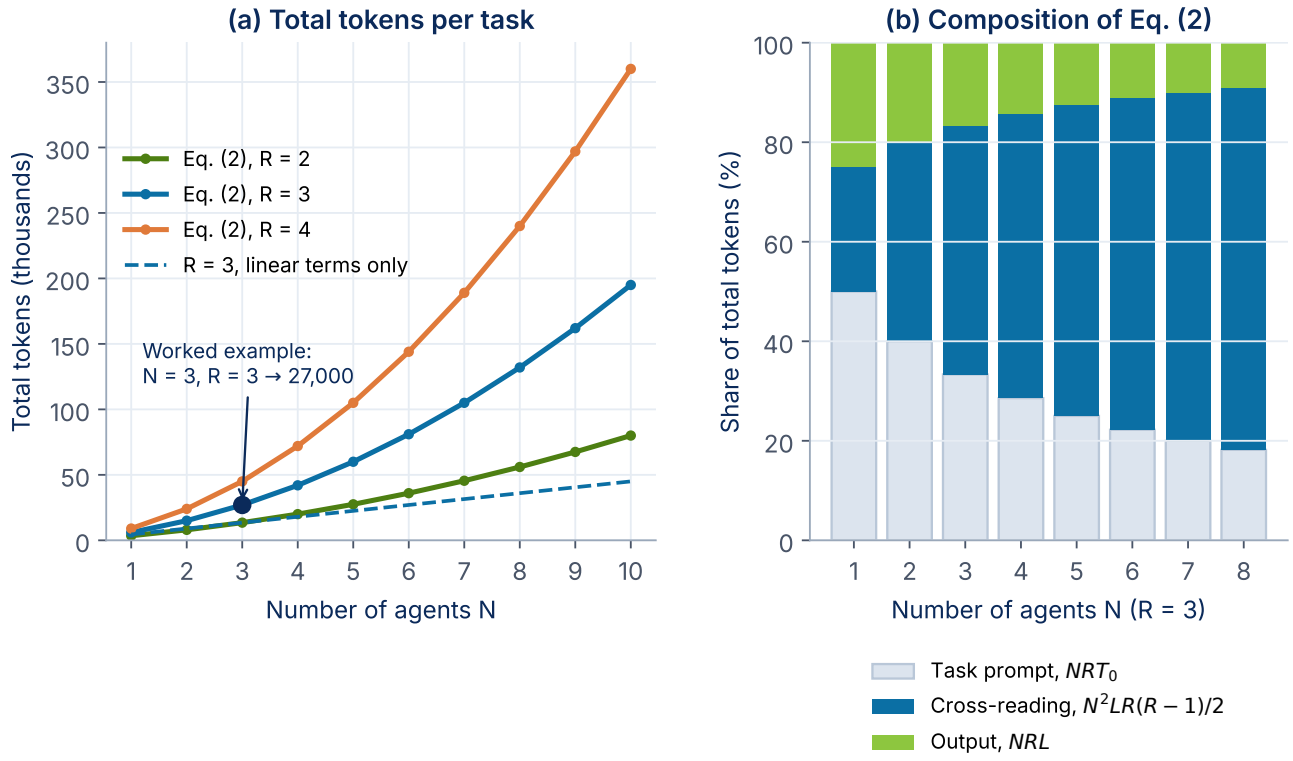


Figure 2. Token multiplication in broadcast debate from Eq. (2) with T_0 = 1,000 and L = 500 tokens. (a) Total tokens against the number of agents N for R = 2, 3 and 4 rounds, with the linear terms alone for R = 3 shown dashed. (b) Share of each term of Eq. (2) at R = 3.

Author's calculation from Eq. (2).

7.2 Latency

For a sequential pipeline of k agents with individual latencies l_i, end-to-end latency is the sum of the l_i. For an orchestrator-worker design with parallel workers,

$$L_{parallel} = l_{plan} + \max_i l_i + l_{synth} \quad (3)$$

where l_{plan} and l_{synth} are the latencies of the orchestrator's planning and synthesis calls. Parallelism therefore reduces latency only when worker tasks are genuinely independent and of similar duration; the slowest worker determines the result, and dependencies between workers reintroduce sequential waiting. Anthropic noted that tasks with many dependencies between agents, or requiring all agents to share the same context, are poor fits for multi-agent designs [13].

7.3 Compounding error

Consider a sequential pipeline of k stages in which stage *i* produces a correct output with probability p_i, given correct input, and assume errors are not corrected downstream. The probability that the pipeline produces a correct final result is

$$P_{\text{success}} = \Pi_{i=1}^{k} p_i \tag{4}$$

With $p_i = 0.95$ at every stage, Eq. (4) gives 0.774 for five stages and 0.599 for ten. Verification stages can arrest the decline. If a verifier detects a stage error with probability δ and a repair attempt then succeeds with probability ρ, the effective stage reliability becomes

$$p'_i = p_i + (1 - p_i)\,\delta\,\rho \tag{5}$$

For $p_i = 0.95$, $\delta = 0.8$ and $\rho = 0.7$, $p'_i = 0.978$ and a five-stage pipeline succeeds with probability of approximately 0.895. Eq. (5) ignores false alarms that reject correct outputs and the correlation between a stage's error and the verifier's failure to notice it, which is likely when both are instances of the same model. This is consistent with the evidence that intrinsic self-correction is weak [23] and that incorrect or incomplete verification is a prominent failure category [3]: verifiers are most valuable when grounded in external signals. Figure 3 extends these examples to longer pipelines and shows that the benefit of a verifier grows with the number of stages.

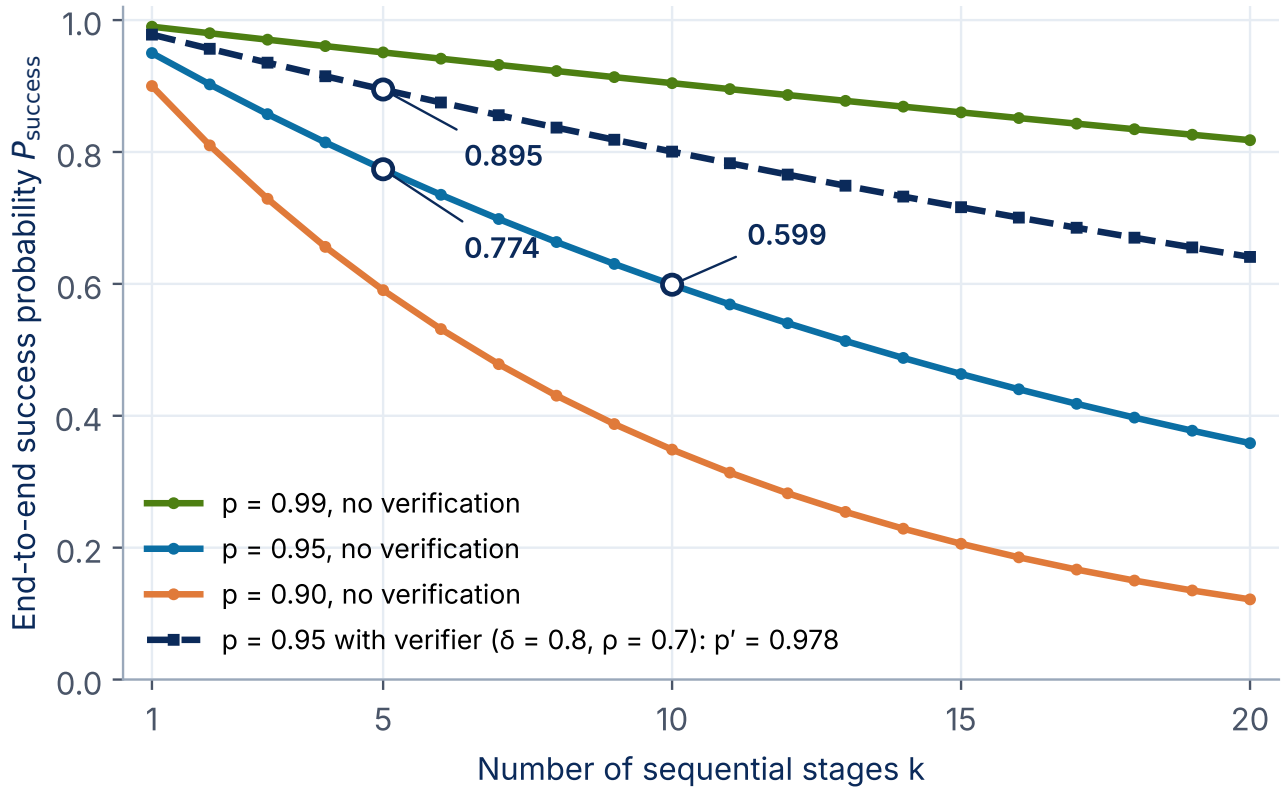


Figure 3. End-to-end success probability of a sequential pipeline from Eq. (4) for per-stage reliabilities of 0.99, 0.95 and 0.90, and for p = 0.95 with a verifier (δ = 0.8, ρ = 0.7) from Eq. (5). Marked points are the worked values of Section 7.3. Errors are assumed independent and verifier false alarms are ignored.

Author's calculation from Eqs. (4) and (5).

8. Security

Multi-agent systems inherit the prompt-injection exposure of single LLM applications and add new paths. Greshake et al. showed that indirect prompt injection, in which instructions are planted in content the system will retrieve, can achieve data theft, manipulation of functionality and control over which APIs are called, and they described worm-like propagation as a threat [25]. In a multi-agent system, the output of one agent is the input of another, so an injection that compromises a single agent, for example a web-browsing worker, can be passed onward as apparently trusted internal communication to agents that hold more privileged tools.

Lee and Tiwari demonstrated this risk experimentally with Prompt Infection, an attack in which malicious prompts self-replicate across interconnected agents in the manner of a computer virus, enabling data theft, scams, misinformation and system-wide disruption; they found

multi-agent systems highly susceptible even when agents did not publicly share all communications, and proposed a defense, LLM Tagging, that in combination with existing safeguards significantly reduced the spread of infection [26]. The OWASP Top 10 for LLM Applications 2025 lists prompt injection first and identifies excessive agency, granting an LLM system more functionality, permissions or autonomy than needed, as a separate risk [27], which multi-agent designs aggravate when privileges are aggregated across agents. Where agents reach tools through the Model Context Protocol, the specification's guidance that tool descriptions be treated as untrusted unless obtained from a trusted server and that hosts obtain explicit user consent before invoking tools applies to every agent in the system [28].

Defensive design follows from treating every inter-agent message as untrusted data. Practical measures include tagging the provenance of every message and retrieved passage; segregating agents that read external content from agents that hold write-capable tools; passing structured data rather than free text across trust boundaries where possible; validating tool arguments against schemas and policy; requiring human approval for irreversible actions; and limiting the number of steps, messages and tokens any agent may consume.

> In a multi-agent system, a message from another agent is not an instruction from the user. Privilege boundaries must be enforced by the orchestration layer, not by asking agents to distrust one another.

9. When Not to Use Multiple Agents, and Design Guidelines

9.1 When a single agent is preferable

The evidence reviewed above supports a default of a single agent with good prompts, tools and retrieval, departing from it only for a demonstrated reason. A multi-agent design is unlikely to pay for itself in the following situations:

- The task fits comfortably within one context window and does not decompose into independent subtasks, so that parallelism offers no benefit [13].
- Subtasks are tightly coupled or require shared context, so that agents must continually exchange state [13].

- A strong single-agent prompt with demonstrations, or a sampling-and-voting ensemble, already matches multi-agent performance on a representative evaluation set [22, 21].
- No external verification signal is available, so that critic and reviewer agents provide only intrinsic feedback of limited value [23].
- The value of a task does not justify a large multiple of single-agent token cost and latency, as estimated by Eq. (2) and Eq. (3).
- The workflow is a fixed, well-understood process better expressed as deterministic code calling a model for bounded subtasks.

9.2 Design guidelines

When a multi-agent design is justified, the following guidelines, derived from the failure taxonomy and the models of Section 7, reduce risk:

1. Begin with the simplest topology that exploits the task's structure, usually an orchestrator with isolated workers for parallelizable work or a short pipeline for staged work, and add agents only when an evaluation shows a gain over the baselines of Section 5.
2. Write explicit role specifications, task objectives and termination conditions for each agent, addressing the system design failures of category FC1 [3].
3. Pass structured artifacts with defined schemas between agents, as MetaGPT does with standardized outputs, rather than unconstrained conversation [2].
4. Place verification at stage boundaries and ground it in external signals such as tests, retrieved evidence or business rules, applying Eq. (5) to judge where verification is worth its cost.
5. Bound every loop with limits on rounds, steps, tokens and wall-clock time, and record full traces for diagnosis using a taxonomy such as MAST.
6. Enforce least privilege per agent and treat all inter-agent messages as untrusted, as described in Section 8 [27, 26].
7. Use heterogeneous models or information sources where diversity is the purpose, since identical instances share failure modes and reduce the benefit predicted by Eq. (1).

10. Conclusion

LLM-based multi-agent systems revive ideas from classical multi-agent research, including task allocation by contract, shared blackboards and organizational roles, using natural language as the medium of coordination [4, 5, 6]. Frameworks such as AutoGen, CAMEL, MetaGPT and ChatDev have made these systems easy to build [1, 16, 2, 17], and debate, role specialization and parallel exploration have produced measurable gains in specific settings [14, 13].

The evidence is nonetheless mixed. Strong single agents and simple ensembles often match multi-agent discussion [22, 21], token consumption grows quickly with the number of agents and rounds, reliability compounds multiplicatively along pipelines, and empirical analysis has identified fourteen recurring failure modes concentrated in system design, inter-agent misalignment and verification [3]. Agent-to-agent prompt injection adds a security dimension absent from single-agent systems [26]. Multi-agent architecture should therefore be treated as an engineering choice justified by task structure and measured against strong baselines, with explicit roles, structured hand-offs, externally grounded verification, bounded loops and enforced privilege boundaries.

References

1. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155. arxiv.org/abs/2308.08155
2. Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., & Schmidhuber, J. (2023). MetaGPT: Meta programming for a multi-agent collaborative framework. arXiv preprint arXiv:2308.00352. arxiv.org/abs/2308.00352
3. Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., & Stoica, I. (2025). Why do multi-agent LLM systems fail? arXiv preprint arXiv:2503.13657. arxiv.org/abs/2503.13657
4. Wooldridge, M., & Jennings, N. R. (1995). Intelligent agents: Theory and practice. The Knowledge Engineering Review, 10(2), 115–152. 10.1017/S0269888900008122
5. Smith, R. G. (1980). The contract net protocol: High-level communication and control in a distributed problem solver. IEEE Transactions on Computers, C-29(12), 1104–1113. 10.1109/TC.1980.1675516

6. Erman, L. D., Hayes-Roth, F., Lesser, V. R., & Reddy, D. R. (1980). The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty. ACM Computing Surveys, 12(2), 213–253. 10.1145/356810.356816

7. Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J.-R. (2023). A survey on large language model based autonomous agents. arXiv preprint arXiv:2308.11432. arxiv.org/abs/2308.11432

8. Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., et al. (2023). The rise and potential of large language model based agents: A survey. arXiv preprint arXiv:2309.07864. arxiv.org/abs/2309.07864

9. Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., & Zhang, X. (2024). Large language model based multi-agents: A survey of progress and challenges. arXiv preprint arXiv:2402.01680. arxiv.org/abs/2402.01680

10. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. International Conference on Learning Representations (ICLR). arXiv:2210.03629. arxiv.org/abs/2210.03629

11. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. arXiv preprint arXiv:2303.11366. arxiv.org/abs/2303.11366

12. Park, J. S., O'Brien, J., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23), 1–22. 10.1145/3586183.3606763

13. Hadfield, J., Zhang, B., Lien, K., Scholz, F., Fox, J., & Ford, D. (2025, June 13). How we built our multi-agent research system. Anthropic Engineering. www.anthropic.com/engineering/multi-agent-research-system

14. Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2023). Improving factuality and reasoning in language models through multiagent debate. arXiv preprint arXiv:2305.14325. arxiv.org/abs/2305.14325

15. Liang, T., He, Z., Jiao, W., Wang, X., Wang, Y., Wang, R., Yang, Y., Shi, S., & Tu, Z. (2024). Encouraging divergent thinking in large language models through multi-agent debate. Proceedings of EMNLP 2024. arXiv:2305.19118. arxiv.org/abs/2305.19118

16. Li, G., Hammoud, H. A. A. K., Itani, H., Khizbullin, D., & Ghanem, B. (2023). CAMEL: Communicative agents for "mind" exploration of large language model society. Advances in Neural Information Processing Systems, 36. arXiv:2303.17760. arxiv.org/abs/2303.17760

17. Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., & Sun, M. (2024). ChatDev: Communicative agents for software

development. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024). arXiv:2307.07924. arxiv.org/abs/2307.07924

18. Chen, W., Su, Y., Zuo, J., Yang, C., Yuan, C., Chan, C.-M., Yu, H., Lu, Y., Hung, Y.-H., Qian, C., Qin, Y., Cong, X., Xie, R., Liu, Z., Sun, M., & Zhou, J. (2023). AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. arXiv preprint arXiv:2308.10848. arxiv.org/abs/2308.10848
19. Zhuge, M., Wang, W., Kirsch, L., Faccio, F., Khizbullin, D., & Schmidhuber, J. (2024). Language agents as optimizable graphs. arXiv preprint arXiv:2402.16823. arxiv.org/abs/2402.16823
20. Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. International Conference on Learning Representations (ICLR). arXiv:2203.11171. arxiv.org/abs/2203.11171
21. Li, J., Zhang, Q., Yu, Y., Fu, Q., & Ye, D. (2024). More agents is all you need. Transactions on Machine Learning Research. arXiv:2402.05120. arxiv.org/abs/2402.05120
22. Wang, Q., Wang, Z., Su, Y., Tong, H., & Song, Y. (2024). Rethinking the bounds of LLM reasoning: Are multi-agent discussions the key? arXiv preprint arXiv:2402.18272. arxiv.org/abs/2402.18272
23. Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., & Zhou, D. (2024). Large language models cannot self-correct reasoning yet. International Conference on Learning Representations (ICLR). arXiv:2310.01798. arxiv.org/abs/2310.01798
24. Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track. arXiv:2306.05685. arxiv.org/abs/2306.05685
25. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), 79–90. 10.1145/3605764.3623985
26. Lee, D., & Tiwari, M. (2024). Prompt infection: LLM-to-LLM prompt injection within multi-agent systems. arXiv preprint arXiv:2410.07283. arxiv.org/abs/2410.07283
27. OWASP Foundation. (2024). OWASP Top 10 for LLM Applications 2025. OWASP Gen AI Security Project. genai.owasp.org/llm-top-10/
28. Model Context Protocol. (2025). Specification, version 2025-06-18. modelcontextprotocol.io. modelcontextprotocol.io/specification/2025-06-18

How to cite this article

Lee, Q. F. (2026). Multi-Agent AI Systems: Architectures, Coordination Patterns and Failure Modes. *Chmltech Online Journal*, *104*(2). https://www.chmltech.com/post/multi-agent-ai-systems-architectures-coordination-and-failure-modes

Ready to find your next improvement?

Book a strategy call. We will talk through your goals, pain points and where AI, systems or process work can make the biggest difference.

Book a Strategy Call

Or email **info@chmltech.com**